

Computational Thinking & Coding —Week 3 Course Material

Computational Thinking & Coding ·Week 3 course material ·Seoul Theological University

One 3-hour session per week × 15 weeks ·All materials in English ·For non-major beginners Source curriculum: MIT OCW 6.100L *Introduction to CS and Programming Using Python* (Fall 2022, Ana Bell) — Lectures 5–6, Recitation 3 **Week 3 scope:** floating point and binary ·approximation with `epsilon` ·bisection search ·writing your own functions **Bring to class:** the pre-class code you ran (§3) and your guess for `num_guesses`

Learning objectives — what you can do at the end of Week 3

#	Objective
1	Say why <code>0.1 + 0.2 == 0.3</code> is False in one sentence, using the words <i>binary</i> and <i>approximation</i>
2	Replace an exact test with an epsilon test and choose a sensible epsilon
3	Trace <code>low</code> , <code>high</code> , <code>ans</code> through two passes of a bisection loop by hand
4	Explain why bisection needs about $\log_2 N$ steps instead of N
5	Write a function with parameters and a <code>return</code> , and say how <code>return</code> differs from <code>print</code>
6	Predict what happens to the number of guesses when <code>epsilon</code> or <code>increment</code> changes

1. What we learn this week [note §...]

Computers store numbers in binary, so some decimals like `0.1` can only be stored **approximately** — tiny errors that grow when repeated. So instead of demanding exact answers, we find “close enough” ones. Then we meet a much faster way to search: like a lecturer finding a student in an alphabetically-seated hall by asking the middle person’s name and **dismissing half the room each time** — that is bisection search.

2. Terms to know [note §...]

Term	One-line definition
floating point	How Python stores real numbers: a pair of integers (significant digits and a base-2 exponent) — an approximation
binary representation	Writing numbers as sums of powers of 2 using only bits 0 and 1, which is what computer hardware stores
epsilon	The small allowed distance to the true answer: stop when $ \text{guess}^2 - x < \text{epsilon}$ (“close enough”)
approximation method	Like guess-and-check, except we increment the guess by a small float and stop when close enough, since exact is impossible
bisection search	Guess the midpoint of an interval, learn if the answer is higher or lower, and cut the search space in half each time

Term	One-line definition
logarithmic growth	Bisection needs about $\log_2 N$ steps versus N steps for one-by-one search — 30 guesses instead of 23 million for 54321

3. Live coding — approximation method [note §...]

From Lecture 5, “Implementation” of the approximation method (slide 26). Predict the output before you run it.

```
x = 36
epsilon = 0.01
num_guesses = 0
guess = 0.0
increment = 0.0001
while abs(guess**2 - x) >= epsilon:
    guess += increment
    num_guesses += 1
print('num_guesses =', num_guesses)
print(guess, 'is close to square root of', x)
```

★ Two questions for the board (from the preview sheet):

- Will it print exactly 6?
- Roughly how many guesses do you expect? The lecture notes report **about 60,000** — why so many?

Board work — fill in during class:

	value
increment	
how far the guess must travel ($0 \rightarrow$ about 6)	
guesses $\approx$ distance $\div$ increment	
what the printed guess looks like (exact 6 or not)	

4. Why the count is what it is [note §...]

★ This section is **the instructor’s framing**, not text from the source sheets: the approximation method walks from 0.0 toward the answer in steps of increment, so the number of steps is roughly $\text{distance} \div \text{increment}$. Halving the increment doubles the work. Bisection, by contrast, **halves the remaining interval every pass**, which is where $\log_2 N$ comes from.

5. Lab B (recitation-style, 40 min) — ground rule [note §...]

The TA never gives away answers — the TA only asks tracing questions.

Warm-up (about 5 min)

Original Problem 2: a function to check whether a number falls in a given range. Fill in the blanks, then predict the two printed lines before running: _____ / _____

```
def check_in_range(x, start, end):
    if x ____ start and x <= ____:      # (1) comparison  (2) variable name
        return "Yes"
    else:
        ____ "No"                      # (3) keyword that sends a value back

print(check_in_range(3, 1, 5))
print(check_in_range(3, 5, 7))
```

6. Core exercises (about 30 min) — Exercise 1: fourth root by bisection search (original Problem 1) (note §...)

(a) **Worked example** — read the original solution, then run it with input 16.

```
x = float(input("Using bisection search calculate the forth root of: "))
epsilon = 0.01
low = 0
high = x
ans = (high + low) / 2
while abs(ans**4 - x) >= epsilon:
    if ans**4 > x:
        high = ans
    else:
        low = ans
    ans = (high + low) / 2
print("Forth root of " + str(x) + " is approximately " + str(ans))
```

(b) **One-line change** — change `epsilon = 0.01` to `epsilon = 0.0001`. Does the loop run more times or fewer? Why?

(c) **Fill in the blank** — complete the search step.

```
low = ____
high = ____
ans = (____ + ____ ) / 2
while abs(ans**4 - x) >= epsilon:
    if ans**4 > x:
        ____ = ans          # which bound moves here?
    else:
        low = ans
```

TA tracing questions

1. “For input 16, what are `low`, `high`, and `ans` right after the first pass of the loop?”
2. “Why is `abs()` needed in the loop condition — what could go wrong without it?”

★ Board work — trace table for input 16:

pass	low	high	ans	ans**4 vs x
1				
2				
3				

7. Exercise 2 — perfect numbers (original Problem 3) (note §...)

(a) **Worked example** — a perfect number equals the sum of its proper divisors.

```
def perfect_number(n):
    my_sum = 0
    for x in range(1, n):
        if n % x == 0:
            my_sum += x
    return my_sum == n

print(perfect_number(6))
```

(b) One-line change — change the test call to `perfect_number(28)`, then `perfect_number(50)`. Predict both outputs before running: _____ / _____

(c) Fill in the blank — complete the function body.

```
for x in range(1, ____):
    if n % x == ____:
        my_sum ____ x
return my_sum ____ n
```

TA tracing questions

1. “For `n = 6`, what is in `my_sum` after the loop finishes checking `x = 3`?”
2. “The function has no `print` — what kind of value does `return my_sum == n` send back?”

8. Exercise 3 — fourth root by incremental approximation (original Problem 4) [note §...]

(a) Worked example — the key loop of the original solution (initial parameters: `epsilon = 0.01`, `ans = 0.0`, `increment = 0.001`, `num_guesses = 0`; negative input is rejected first).

```
while abs(number - (ans**4)) >= epsilon and ans**4 <= number:
    ans += increment
    num_guesses += 1
```

(b) One-line change — change `increment = 0.001` to `increment = 0.1`. What happens to `num_guesses`, and can the search now **miss** the answer? _____

(c) Fill in the blank — complete the loop.

```
while abs(number - (ans**4)) >= epsilon and ans**4 <= ____:
    ans += ____
    num_guesses += 1
```

TA tracing questions

1. “After 10 passes of the loop, what exactly is stored in `ans`?”
2. “Which of the two and conditions stops the loop when the guess overshoots the target?”

9. Self-check (about 5 min) [note §...]

- I can trace `low`, `high`, `ans` through two passes of a bisection loop by hand.
- I can explain the difference between `return` and `print`, and what a Boolean return value is.
- I can explain the trade-off between a smaller `increment / epsilon` and the number of iterations.

10. Questions to think about [note §...]

1. “Never use `==` to test floats.” Where else in life do we accept “close enough” instead of exact equality, and how do we choose the tolerance (our `epsilon`)?
2. Bisection search only works when the answers have an ordering and you get feedback like “too high / too low.” Think of a guessing situation where you get **no such feedback** — why does halving no longer help there?